

Multilevel queue scheduling:

```
#include <stdio.h>
#include <string.h>

#define MAX 100

typedef struct {
    int pid;
    int arrival;
    int burst;
    int completion;
    int turnaround;
    int waiting;
    char type[10]; // "System" or "User"
} Process;

int main() {
    Process sysQ[MAX], userQ[MAX];
    int n, i, sCount = 0, uCount = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    Process temp;

    // Input
    for(i = 0; i < n; i++) {
        printf("\nProcess %d\n", i + 1);
        printf("Enter Process ID: ");
        scanf("%d", &temp.pid);

        printf("Enter Arrival Time: ");
        scanf("%d", &temp.arrival);

        printf("Enter Burst Time: ");
        scanf("%d", &temp.burst);

        printf("Enter Type (System/User): ");
        scanf("%s", temp.type);

        if(strcmp(temp.type, "System") == 0) {
            sysQ[sCount++] = temp;
        }
    }
}
```

```

    } else {
        userQ[uCount++] = temp;
    }
}

int current_time = 0;

// --- Execute System Queue (Higher Priority) ---
for(i = 0; i < sCount; i++) {

    if(current_time < sysQ[i].arrival) {
        current_time = sysQ[i].arrival;
    }

    current_time += sysQ[i].burst;
    sysQ[i].completion = current_time;

    sysQ[i].turnaround = sysQ[i].completion - sysQ[i].arrival;
    sysQ[i].waiting = sysQ[i].turnaround - sysQ[i].burst;
}

// --- Execute User Queue ---
for(i = 0; i < uCount; i++) {

    if(current_time < userQ[i].arrival) {
        current_time = userQ[i].arrival;
    }

    current_time += userQ[i].burst;
    userQ[i].completion = current_time;

    userQ[i].turnaround = userQ[i].completion - userQ[i].arrival;
    userQ[i].waiting = userQ[i].turnaround - userQ[i].burst;
}

// --- Output ---
float totalWT = 0, totalTAT = 0;

printf("\n--- System Processes ---\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < sCount; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        sysQ[i].pid,

```

```

        sysQ[i].arrival,
        sysQ[i].burst,
        sysQ[i].completion,
        sysQ[i].turnaround,
        sysQ[i].waiting);

    totalIWT += sysQ[i].waiting;
    totalTAT += sysQ[i].turnaround;
}

printf("\n--- User Processes ---\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");

for(i = 0; i < uCount; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        userQ[i].pid,
        userQ[i].arrival,
        userQ[i].burst,
        userQ[i].completion,
        userQ[i].turnaround,
        userQ[i].waiting);

    totalIWT += userQ[i].waiting;
    totalTAT += userQ[i].turnaround;
}

int totalProcesses = sCount + uCount;

printf("\nAverage Waiting Time = %.2f", totalIWT / totalProcesses);
printf("\nAverage Turnaround Time = %.2f\n", totalTAT / totalProcesses);

return 0;
}

```

OUTPUT:

```
C:\Users\Nitya\OneDrive\Des X + v
Nitya Swadi, 1BM24CS361
Enter number of processes: 3

Process 1
Enter Process ID: 1
Enter Arrival Time: 0
Enter Burst Time: 3
Enter Type (System/User): System

Process 2
Enter Process ID: 2
Enter Arrival Time: 0
Enter Burst Time: 4
Enter Type (System/User): User

Process 3
Enter Process ID: 3
Enter Arrival Time: 1
Enter Burst Time: 6
Enter Type (System/User): User

--- System Processes ---
PID    AT    BT    CT    TAT    WT
1       0     3     3     3     0

--- User Processes ---
PID    AT    BT    CT    TAT    WT
2       0     4     7     7     3
3       1     6    13    12     6

Average Waiting Time = 3.00
Average Turnaround Time = 7.33

Process returned 0 (0x0)  execution time : 51.008 s
Press any key to continue.
```